

check a number is Spy Number or not.
number is called a Spy Number if, Sum of digits = Product of digits
123 ,1124.

```
1+2+3 = 1*2*3

*/
/*
Spy number logic :
-----
1) Initialize sum = 0 and prod = 1.
2) Extract each digit using % 10 till num!=0
3) Add the digit to sum.
4) Multiply the digit with product.
4) Remove the last digit using / 10.
5) Compare sum and product.
If equal, it is a Spy Number.
*/

void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Is "+number+" a spy number :"+isSpy(number));
}

boolean isSpy(int num) //0
{
    int sum = 0; //6
    int prod = 1; //6

    while(num!=0)
    {
        int digit = num % 10; //digit = 1
        sum = sum + digit;
        prod = prod * digit;
        num = num /10;
    }

    return sum==prod;
}
```

/* Check a number is prime Number or not.

If a number is divisible by 1 and itself (exactly two factors) then It is called prime number.

```
2, 3, 5, 7, 11 -> Prime
1, 4, 6, 9 -> Not Prime
*/

/*
Prime number logic :
-----
1) If the number is less than or equal to 1, it is not prime.
2) Check if the number is divisible by any number from 2 to num/2.
3) If divisible, it is not prime.
4) If no divisors are found, it is prime.
*/

Logic1
-----
void main()
{
    int num = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Is "+num+" a prime number :"+isPrime(num));
}

//Good time complexity
boolean isPrime(int num)
{
    if(num <=1)
    {
        return false;
    }

    for(int i=2; i<=num/2; i++)
    {
        if(num % i==0)
        {
            return false;
        }
    }

    return true;
}

Logic 2 :
-----

void main()
{
    int num = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Is "+num+" a prime number :"+isPrime(num));
}

boolean isPrime(int num)
{
    int count = 0;

    if(num <=1)
    {
        return false;
    }

    for(int i=1; i<=num; i++)
    {
        if(num%i==0)
        {
            count++;
        }
    }

    return count==2;
}
```

/* Check a number is duck Number or not.

A number is called a Duck Number if:
It contains at least one zero (0) and must be positive number
The number should NOT start with zero

```
1708
170
0 -> Not a duck number
*/

/*
Logic for Duck numer :
-----
1) Take the number as input.
2) if number is zero OR negative return false
3) initialize containsDigit = false;
4) Extract each digit using % 10 till num !=0
5) Check if any digit is 0.
6) If a 0 is found, make containsDigit true, use break.
7) Remove the last digit using / 10.
8) return containsDigit
*/

void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Is "+number+" duck number :"+isDuck(number));
}

boolean isDuck(int num)
{
    if(num ==0 || num < 0)
    {
        return false;
    }

    boolean containsDigit = false;

    while(num!=0) //0
    {
        int digit = num % 10; //digit = 1
        if(digit==0)
        {
            containsDigit = true;
            break;
        }
        num = num /10;
    }

    return containsDigit;
}
```

/*Check a number is Armstrong or not.

A number is called an Armstrong number if the sum of each digit raised to the power of the number of digits is equal to the original number.

```
153  370 371 1634

*/

Math class which is available in java.lang package, It is a utility class and provides various static methods.
In order to find the power of the number, Math class has provided a static method called pow()

public static double pow(double x, double y)

Pow.java
-----
void main()
{
    IO.println(Math.pow(2,3)); //8.0

    int x = 3;
    int y = 2;
    double val = Math.pow(x,y); //9.0
    IO.println(val);
}

/*
Logic for Armstrong  number :
-----
1) Initialize totalDigits = 0 and double sum = 0.0. [digits = total number of digits]
2) Store the number in a temporary variable i.e temp
3) count the total digits till num!=0 (totalDigits++)
4) Calculate digit to the power and add it to a sum. [sum = sum + Math.pow(digit, totalDigits);]
5) Remove the last digit using / 10.
6) After processing all digits, compare the sum with the original number.
If equal, it is an Armstrong Number.
*/

void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Is "+number+" strong number :"+ isArmstrong(number));
}

boolean isArmstrong(int num)
{
    double sum = 0;
    int totalDigits = countDigits(num);
    int temp = num;

    while(num!=0)
    {
        int digit = num % 10;
        sum = sum + Math.pow(digit, totalDigits);
        num = num /10;
    }

    return sum==temp;
}

//Count the total number of digits
int countDigits(int num)
{
    int count = 0;

    while(num!=0)
    {
        num = num /10;
        count++;
    }

    return count;
}
```

/* Perfect Number

A perfect number is a positive integer that equals the sum of its proper positive divisors, excluding the number itself.

Example :
a) 6 -> 1, 2 , 3 so [1 + 2 + 3 = 6]
b) 28 -> 1, 2, 4, 7, 14 [1 + 2 + 4 + 7 + 14 = 28]

```
*/

/*
Logic for Perfect number :
-----
1) Store the number in a temp variable
2) Initialize sum = 0.
3) Find all factors from 1 to num/2.
4) If num % i == 0, add i to sum.
5) After the loop, compare sum with temp.
6) If equal, it is a Perfect Number.
*/

void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Is "+number+" perfect number :"+isPerfect(number));
}

boolean isPerfect(int num)
{
    int temp = num;
    int sum = 0;

    for(int i=1; i<=num/2; i++)
    {
        if(num % i==0)
        {
            sum = sum + i;
        }
    }

    return sum == temp;
}
```

/* count all the even numbers in a given digit

```
224 = 3
135 = 0
124 = 2
*/

void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a number :"));
    IO.println("Total even digits are :"+countEvenDigits(number));
}

int countEvenDigits(int num)
{
    int count = 0;

    while(num !=0)
    {
        int digit = num % 10;

        if(digit % 2==0)
        {
            count++;
        }

        num = num /10;
    }

    return count;
}
```

Can we write mulplie methods with same name in a java class